UNIVERSIDAD CHAMPAGNAT

Licenciatura en Sistemas de Información — Programación Distribuida

Trabajo Práctico N° 5: De API REST a Microservicios

Documento de Diseño Arquitectónico, Desacople Funcional y Trabajo Colaborativo

Repositorio Remoto Único:	https://github.com/astorgalucia0-prog/tp5_progdistribuida
Rama Principal / Rama de Trabajo:	`main | docs/arquitectura (Tag: tp5-entrega)`
Integrantes del Grupo:	Lucía Astorga (astorgalucia0@gmail.com) [Grupo N° 5]
Docentes Colaboradores:	Adriel Álvarez (alvarezadriel@uch.edu.ar) • Edgardo Cortez (cortezedgardo@uch.edu.ar)
Versión Funcional de Partida:	API REST Distribuida con Consenso Raft (Cluster 3 Nodos), MongoDB y Redis Lock (TP4)

1. Objetivo y Alcance del Trabajo Práctico

El objetivo central es evolucionar la aplicación existente hacia una **arquitectura basada en microservicios**, incorporando un **flujo de trabajo colaborativo profesional con Git** (repositorio único compartido, ramas de características, mensajes semánticos de commit y Pull Requests). La división responde a **responsabilidades de negocio claras**, baja interdependencia y el principio **Database-per-Service**, evitando divisiones artificiales basadas en una tabla por servicio.

2. Inventario de Funcionalidades y Endpoints Existentes (Sección 5)

Funcionalidad / Endpoint	Responsabilidad Actual	Datos Utilizados	Dependencias
`GET /api`	Diagnóstico de salud general, nodo Raft y estado DB/Redis.	nodeId, role, estado Mongo/Redis, timestamp.	raftNode, database, redis
`GET /api/productos`	Listar catálogo comercial; siembra datos iniciales si la BD está vacía.	Colección 'productos' (MongoDB).	Producto (Mongoose), Mongo
`GET /api/productos/:id`	Consulta individual de ficha y existencias por ID o código.	id / codigo comercial (PROD-XXX).	Producto, MongoDB
`POST /api/productos`	Alta de artículo; propone comando al log Raft si es Leader (o 409).	codigo, nombre, precio, stock, categoria, desc.	raftNode, Producto, Mongo
`PUT /api/productos/:id`	Modificación de atributos comerciales o stock con consenso Raft.	id, campos a actualizar en req.body.	raftNode, Producto, Mongo
`DELETE /api/productos/:id`	Baja física del artículo por quórum en Raft.	id / codigo.	raftNode, Producto, Mongo
`POST /api/productos/:id/reservar`	Exclusión mutua con Redis Lock + decremento de stock por consenso Raft.	id, lockKey 'lock:producto:id', stock.	redis.js, raftNode, MongoDB
`GET /raft/status`	Métricas internas y observabilidad del protocolo de consenso.	nodeId, role, currentTerm, commitIndex, log.	raftNode.getStatus()

3. Identificación de Acoplamientos y Propuesta de Desacople (Sección 7)

Caso	Acoplamiento Detectado	Problema Generado	Propuesta de Desacople
1	**Catálogo y Stock unificados en Producto.js**	Contención entre lecturas masivas de catálogo y escrituras transaccionales de stock. Imposibilidad de escalar independientemente.	Separar en dos microservicios: **Catalog Service** (datos descriptivos) e **Inventory Service** (existencias y reservas).
2	**Lógica de Concurrencia (Redis Lock) embebida en HTTP**	El controlador HTTP orquesta primitivas Redis y timeouts de locking. Si cambia la infraestructura de bloqueo, se quiebra la API REST.	Encapsular Redis como detalle interno de **Inventory Service**, exponiendo la interfaz REST limpia POST /inventory/reserve.
3	**Dependencia Circular entre Rutas y Consenso Raft**	service.js exporta raftNode hacia endpoints.js, y este último se importa en el servidor. Una falla en Raft tumba toda la API.	Aislar cada microservicio en procesos Node.js autónomos con ciclo de vida propio y comunicación desacoplada vía HTTP/REST.

4. Propuesta y Fichas Técnicas de Microservicios (Sección 8)

Microservicio	Responsabilidad Principal	Persistencia Exclusiva	Endpoints Clave

Catalog Service
Puerto :3001
	Gestión de fichas comerciales, categorías y precios oficiales de lista.	`MongoDB: db_catalog`	`GET /catalog/products`
			`GET /catalog/products/:code`
			`POST /catalog/products`

Inventory Service
Puerto :3002
	Control atómico de existencias físicas, reservas concurrentes y devoluciones.	`MongoDB: db_inventory`
		`Redis (:6379) Locks & TTL`
			`GET /inventory/:code`
			`POST /inventory/reserve`
			`POST /inventory/confirm`

Orders Service
Puerto :3003
	Orquestación del flujo de compras, cálculo de totales y trazabilidad de pedidos.	`MongoDB: db_orders`	`POST /orders`
			`GET /orders/:id`
			`POST /orders/:id/confirm`

API Gateway
Puerto :8080
	Punto único perimetral de acceso, enrutamiento por prefijo, CORS y seguridad.	N/A (Stateless)	`/api/catalog/* → :3001`
			`/api/inventory/* → :3002`
			`/api/orders/* → :3003`

5. Matriz de Propiedad y Acceso a los Datos (Sección 9)

Regla de Diseño Estricta: Ningún servicio puede modificar directamente la persistencia de otro servicio. Toda interacción debe efectuarse a través de la interfaz pública REST.

Dato / Recurso	Servicio Propietario	Quién Consulta	Quién Modifica	Forma de Acceso
Ficha de Producto	**Catalog Service**	Clientes, Orders, Inv.	Catalog Service	`GET /catalog/products/:code`
Precio Oficial Lista	**Catalog Service**	Orders Service, Clientes	Catalog Service	`GET /catalog/products/:code`
Stock Disponible	**Inventory Service**	Orders Service, Clientes	Inventory Service	`GET /inventory/:code`
Reserva de Stock	**Inventory Service**	Orders Service	Inventory Service	`POST /inventory/reserve`
Orden de Compra	**Orders Service**	Cliente, Logística	Orders Service	`POST /orders, GET /orders/:id`

6. Comunicaciones Inter-Servicio y Trazado de Casos de Uso (Sección 10)

Origen	Destino	Necesidad de Negocio	Método / Endpoint	Respuesta Esperada
API Gateway	Catalog Svc	Navegación pública de artículos.	`GET /catalog/products`	200 OK [items]
Orders Svc	Catalog Svc	Verificar existencia y fijar precio acordado.	`GET /catalog/products/:code`	200 OK {price, name}
Orders Svc	Inventory Svc	Reserva atómica temporal de unidades.	`POST /inventory/reserve`	200 OK {res_id} / 409 Conflict
Orders Svc	Inventory Svc	Confirmar deducción de stock tras el pago.	`POST /inventory/confirm`	200 OK {stock_deducted: true}

Trazado del Caso de Uso Principal (Creación de Orden con Reserva de Stock):

1. **Cliente** → POST /api/orders { customer_id: 'CLI-1', items: [{ code: 'PROD-001', qty: 2 }] }.
2. **Orders Service** → GET /api/catalog/products/PROD-001: Consulta a Catalog Service para validar código y precio unitario ($45.000).
3. **Orders Service** → POST /api/inventory/reserve { order_id, items }: Inventory Service toma Lock Redis, descuenta 2 de stock disponible, incrementa stock reservado y responde con reservation_id: 'RES-501'.
4. **Orders Service** calcula el total monetario ($90.000), persiste la orden en db_orders con estado PENDING y retorna HTTP 201 Created al cliente.

7. Checklist de Validación de Límites Arquitectónicos (Sección 12)

Criterio de Validación	Estado	Fundamentación Técnica
Responsabilidad explicada en una frase clara.	**CUMPLE**	Catalog gestiona fichas y precios; Inventory controla existencias y locks; Orders orquesta compras.
Agrupa reglas de negocio afines.	**CUMPLE**	Cada servicio concentra la lógica exclusiva de su dominio (precios, stock atómico, pedidos).
No fue creado por existir una tabla previa.	**CUMPLE**	Se descompuso la entidad monolítica Producto en dominios desacoplados de Catálogo vs. Almacén.
Interfaz pública sin exponer detalles internos.	**CUMPLE**	JSON REST limpio sobre HTTP. El uso interno de Redis o Mongoose es invisible para el resto.
Modificable internamente sin cambios en cascada.	**CUMPLE**	Cambiar la base de datos de Inventory de Mongo a Postgres no afecta a Catalog ni a Orders.
No duplica reglas de negocio de otra autoridad.	**CUMPLE**	Inventory es la única autoridad en stock; Catalog en precios; Orders en estados de compra.
Dependencias necesarias y justificadas.	**CUMPLE**	Catalog e Inventory son hojas (cero dependencias). Orders solo invoca para validar pedidos.
Límites fuera de alcance explícitos.	**CUMPLE**	Documentados en las fichas de diseño (e.g. Catalog no administra unidades en bodega).

8. Resumen de Entregables de la Etapa (Sección 14)

- **1. Repositorio Remoto Único:** https://github.com/astorgalucia0-prog/tp5_progdistribuida
- **2. Integrantes:** Lucía Astorga (Grupo N° 5)
- **3. Docentes Colaboradores:** Adriel Álvarez (alvarezadriel@uch.edu.ar) y Edgardo Cortez (cortezedgardo@uch.edu.ar)
- **4. Versión Base Funcional:** Preservada en commit inicial d57faec (Cluster Raft 3 nodos con Redis Lock y Mongoose)
- **5. Seguridad y Datos Sensibles:** .gitignore blindado (node_modules, .env, *.log, certs) y .env.example documentado
- **6. Trabajo Colaborativo:** Ramas main y docs/arquitectura, commits semánticos y Pull Request activo en GitHub
- **7 a 15. Documentación Técnica:** Integrada en docs/arquitectura.md, README.md y en el presente informe oficial
- **16. Etiqueta de Entrega:** Tag tp5-entrega creado sobre la versión aprobada.